

1. Propuesta

La propuesta plantea el desarrollo de un sistema de procesamiento de lenguaje natural orientado a la generación automática de resúmenes en lenguaje sencillo (Plain Language Summaries, PLS) a partir de textos clínicos y científicos en salud. El enfoque combina clasificación de lenguaje técnico y sencillo mediante representaciones TF-IDF y embeddings BERT, con modelos generativos tipo decoder de pequeña y mediana escala (<3B de parámetros) LLaMA-3.2-3B, Qwen3-1.7B, Phi-3.5-mini-instruct y Gemma-2B ajustados mediante LoRA e Instruction Tuning.

Además, se integran técnicas de Chain-of-Thought prompting para mejorar la coherencia y razonamiento de los modelos. Los resultados se compararán con modelos comerciales mayores a 10B parámetros (GPT-5, Gemini 2.5 Pro, Grok-4 y Claude Sonnet-4) empleando métricas de relevancia, facticidad y legibilidad.

El sistema final incluirá un prototipo funcional con API (FastAPI) e interfaz de usuario (Streamlit) que combine los clasificadores y generadores, buscando mejorar la alfabetización en salud y facilitar el acceso a información médica comprensible, confiable y accesible para pacientes y público general

2. Datos: recolección y preparación

2.1. Fuentes de datos y recolección

Para la fase de recolección, el conjunto de datos se construyó tomando como base las fuentes publicadas en la investigación "Bridging the Gap in Health Literacy: Harnessing the Power of Large Language Models to Generate Plain Language Summaries from Biomedical Texts" (Arias-Russi, Salazar-Lara, Manrique, 2025).

Los datos provienen específicamente del corpus Cochrane, el cual se encuentra estructurado en dos categorías principales:

- Textos Científicos (non-PLS): Artículos y reportes biomédicos en lenguaje técnico.
- Resúmenes en Lenguaje Sencillo (PLS): Versiones simplificadas de los textos científicos, destinadas a una audiencia general.

Los archivos fuente originales estaban organizados en directorios separados para entrenamiento (train) y prueba (test), y sub-divididos por cada una de las clases (carpetas pls y non_pls).

Durante la etapa de recopilación y análisis de fuentes de datos se identificaron varios inconvenientes, principalmente relacionados con las restricciones de licencia y el tamaño de los conjuntos disponibles. En primer lugar, se evidenció que algunas fuentes ampliamente utilizadas, como *Cochrane*, actualmente mantienen una licencia cerrada que impide el uso de sus artículos

recientes para el entrenamiento de modelos de inteligencia artificial, lo que limita su aprovechamiento en proyectos académicos.

En segundo lugar, aunque se identificaron múltiples repositorios públicos como *BioLaySumm*, estos presentan dificultades prácticas para el fine-tuning debido a su gran tamaño, ya que contienen más de 30.000 registros, muchos de ellos con textos que superan las 10.000 palabras por artículo. Este volumen incrementa de forma significativa el consumo de GPU y memoria VRAM, haciendo el entrenamiento local poco viable.

Por estas razones, se optó finalmente por emplear el dataset disponible en el estudio de **Arias-Russi, Salazar-Lara y Manrique (2025)**, el cual ofrece un equilibrio adecuado entre calidad de los datos y accesibilidad computacional, permitiendo realizar el fine-tuning de manera eficiente dentro de los recursos disponibles.

2.2 Proceso de limpieza y preparación

El proceso de preparación se centró en consolidar los múltiples archivos de texto (.txt) fuente en un formato estructurado y etiquetado, óptimo para el entrenamiento de los modelos.

Normalización y carga (Clasificación): Para la tarea de clasificación, se implementó una función (`convertir_txt_data_frame`) que itera sobre los directorios del corpus. Durante la lectura de cada archivo .txt, se aplicó una normalización ligera:

- Se utilizó codificación UTF-8 y se configuró la omisión de errores de lectura (`errors="ignore"`).
- Se eliminó el espacio en blanco inicial o final de cada texto mediante la función `.strip()`.

A cada texto cargado se le asignó una etiqueta binaria: 1 para los textos PLS y 0 para los textos científicos (non-PLS). Finalmente, todos los conjuntos de datos (entrenamiento y prueba de ambas clases) se concatenaron en un único DataFrame de pandas. Este conjunto consolidado se guardó en fragmentos (chunks) de 15,000 filas para facilitar su manejo y versionamiento en GitHub.

Emparejamiento PLS/Non-PLS (Generación): De forma paralela, para el análisis exploratorio y la futura tarea de generación de resúmenes, se realizó un proceso de emparejamiento. Se desarrolló una función (`convertir_txt_data_frame_test`) que procesa los nombres de archivo para extraer un identificador común (p.ej., eliminando sufijos como `-abstract` o `-pls`).

Usando este identificador, se realizó un merge de los DataFrames para alinear cada texto científico (article) con su resumen PLS (summary) correspondiente. Esto resultó en un conjunto de datos (`cochrane_total`) donde cada fila contiene el par de textos alineados, fundamental para el fine-tuning de los modelos generativos.

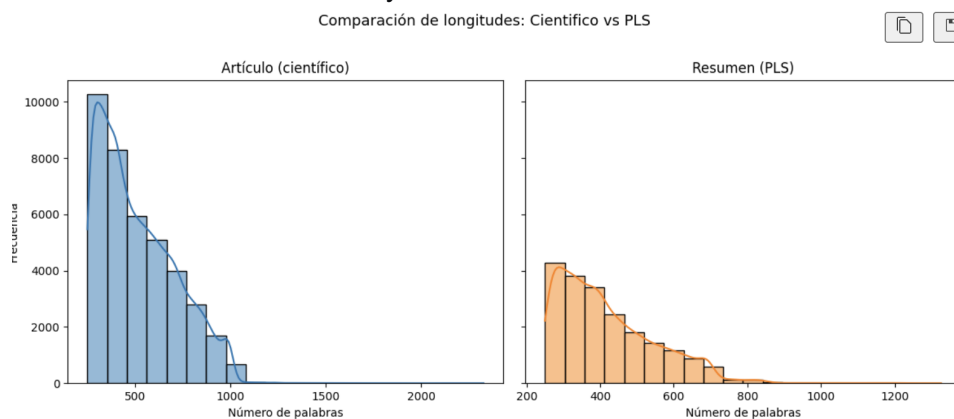
2.3 Estadísticas exploratorias (EDA)

El Análisis Exploratorio de Datos (EDA) inicial se enfocó en cuantificar los documentos y analizar sus distribuciones de longitud, un factor clave para la clasificación.

Conteo de documentos: El conteo total de archivos de texto cargados desde el corpus Cochrane, antes de la consolidación, fue el siguiente:

- Entrenamiento (Científico / non-pls): 31,118
- Entrenamiento (PLS): 16,241
- Prueba (Científico / non-pls): 7,664
- Prueba (PLS): 3,940

Análisis de longitud: Se realizó un análisis comparativo de la longitud (número de palabras) entre las dos clases de textos. Como se evidencia en los histogramas generados, existe una diferencia de distribución muy marcada entre ambas clases.



Los textos científicos (izquierda) muestran una distribución de longitud mucho más amplia, con una media considerablemente mayor y una cola larga que indica la presencia de documentos muy extensos (superando frecuentemente las 2000 palabras).

En contraste, los resúmenes PLS (derecha) son significativamente más cortos y presentan una distribución mucho más acotada y consistente, agrupándose la gran mayoría por debajo de las 500 palabras.

Esta clara separación en la longitud es un indicador robusto que, como se verá en los resultados (Sección 4), facilita que los modelos, incluso los dispersos como TF-IDF, logren una alta precisión en la tarea de clasificación.

3. Descripción del proceso de construcción de los modelos o solución

3.1 Construcción del clasificador:

Se abordaron dos aproximaciones para construir el clasificador: un modelo disperso basado en representaciones TF-IDF y un modelo contextual basado en embeddings. Ambos modelos fueron entrenados y evaluados sobre el mismo conjunto de datos (divididos en entrenamiento, pruebas y validación)

3.1.1 Modelo Disperso (TF-IDF + Clasificadores Lineales)

Para el enfoque disperso se utilizó un pipeline con vectorización TF-IDF y una búsqueda de hiperparámetros mediante GridSearchCV. El texto fue previamente normalizado con un pre-procesador personalizado (TextPreprocessor) que incluyó limpieza ligera y tokenización controlada. Se probaron múltiples algoritmos supervisados como: RidgeClassifier, SGDClassifier, PassiveAggressiveClassifier, LinearSVC, Perceptron, LogisticRegression, RandomForest, ComplementNB y MultinomialNB, para la evaluación utilizaron las métricas: Accuracy, Precision, Recall, F1-score.

3.1.2 Modelo Contextual (ELECTRA Small Discriminator)

Para el enfoque empleó un modelo contextual de tipo transformer, concretamente google/electra-small-discriminator de aproximadamente 14m de parámetros.

Para el ejercicio se utilizó el tokenizador del modelo y un pipeline para trabajar con textos largos que los convierte y los fragmenta respetando la ventana de contexto máxima de 512 tokens. Encima se define una cabeza ligera para clasificación binaria. El entrenamiento se realiza por épocas con dataloaders

Este modelo utiliza un enfoque de entrenamiento más eficiente que BERT, en la evaluación del rendimiento de Clark et al. (2020) demostraron que ELECTRA-Small alcanza resultados comparables a BERT-Base en el GLUE Benchmark (Wang et al., 2019), a una fracción del costo computacional.

3.2 Finetuning modelos

Hasta el momento se han realizado procesos de fine-tuning sobre tres modelos de pequeña/media escala: *gemma3-1b-pt*, *llama3.2-1b* y *qwen3-1p7b*. El ajuste se hizo usando técnicas de adaptación ligera (LoRA/PEFT), con tokenización y fragmentación por ventana de contexto, optimización por mini-lotes, y checkpoints periódicos para garantizar recuperación. Los experimentos se registraron (logs y métricas) y se preservaron los mejores pesos según la métrica de validación. El objetivo fue adaptar cada decodificador al estilo PLS manteniendo eficiencia en consumo de GPU y memoria.

Durante esta primera iteración del proceso de *fine-tuning*, el objetivo principal fue adaptar el código y los parámetros de entrenamiento para su ejecución en un entorno local con una GPU NVIDIA RTX 4050 de 6 GB de VRAM. Esto implicó ajustar el tamaño de lote, la longitud máxima de secuencia y la estrategia de acumulación de gradientes para optimizar el uso de memoria y garantizar la estabilidad del entrenamiento.

Para la entrega final, se planea migrar el entrenamiento a un entorno con GPU A100 disponible en Google Colab, lo que permitirá escalar los experimentos, ampliar el tamaño del conjunto de datos y aprovechar la mayor capacidad de cómputo para realizar un ajuste más profundo y eficiente de los modelos.

4. Resultados obtenidos.

4.1. Métricas de evaluación

Para los modelos asociados a la tarea Clasificación de lenguaje técnico vs. sencillo se utilizarán las métricas Accuracy, Precision, Recall, F1-score.

Para los modelos asociados a la generación de texto sencillo, incluyendo menores a 3B parámetros y comerciales superiores a 10B, se utilizarán las métricas de Relevancia (Zhang et al., 2020), Factualidad (Zha et al., 2023) y Legibilidad (Flesch, 1948; Coleman & Liao, 1975; Gunning, 1952; Chall & Dale, 1995).

4.2. Resultados clasificador

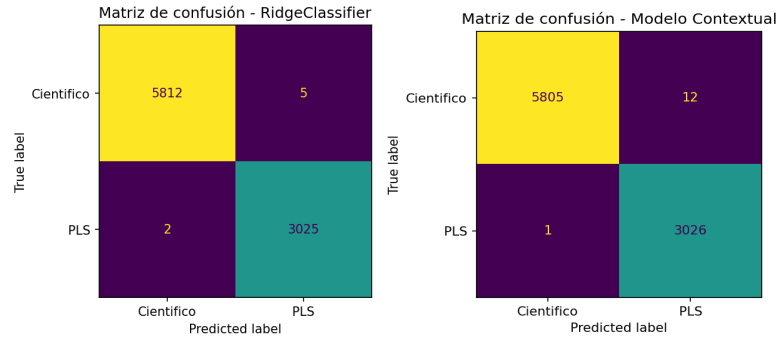
Los mejores resultados para el clasificador disperso se obtuvieron con el algoritmo RidgeClassifier (luego de un exhaustivo grid search) configurado con *solver = sag* y *alpha = 0.5*, alcanzando una exactitud de 0.9992. Este desempeño refleja una alta estabilidad y consistencia entre las particiones de validación cruzada, evidenciando que las representaciones TF-IDF capturan de manera efectiva las diferencias léxicas y terminológicas entre los textos científicos y los resúmenes en lenguaje sencillo.

Por su parte, el modelo contextual, basado en *ELECTRA Small Discriminator*, alcanzó una exactitud de 0.9985, ligeramente inferior al clasificador disperso, aunque manteniendo un nivel de rendimiento sobresaliente. Estos resultados confirman la robustez de ambos enfoques y la clara separabilidad entre las clases analizadas.

Los resultados detallados se presentan a continuación:

Modelo	RidgeClassifier		Modelo Contextual	
Métrica	Clase 0 (Científico)	Clase 1 (PLS)	Clase 0 (Científico)	Clase 1 (PLS)
precision	0.9997	0.9983	0.9998	0.9961
recall	0.9991	0.9993	0.9979	0.9997
f1-score	0.9994	0.9988	0.9989	0.9979

Matriz de confusion:



El tiempo de entrenamiento y predicción fue considerablemente menor en el modelo basado en RidgeClassifier en comparación con los modelos contextuales, lo que permitió realizar ejecuciones rápidas incluso sobre volúmenes amplios de texto. En contraste, el modelo contextual presentó tiempos de entrenamiento e inferencia significativamente más altos, debido a la complejidad computacional inherente a las arquitecturas tipo *Transformer* y al procesamiento de embeddings contextuales. Esta diferencia resalta la eficiencia del enfoque disperso para tareas de clasificación binaria con alto volumen de datos y recursos limitados.

4.3. Resultados Fine tuning

Modelo	Precision	Recall	F1-score	Factualidad (AlignScore)	Legibilidad (Flesch)	Nº Palabras (prom.)
Gemma-3-1B-PT (FT)	0.8211	0.8139	0.8173	0.283	40.56	320.6
Llama-3.2-1B (FT)	0.8279	0.8130	0.8203	0.328	43.18	334.7
Qwen-3-1.7B (FT)	0.8174	0.8263	0.8217	0.342	29.24	793.3
Claude Sonnet 4.5 (API)	0.8384	0.8276	0.8329	0.525	28.45	304.5
Grok 4 (API)	0.8357	0.8364	0.8360	0.543	31.90	365.5

El score de legibilidad refleja qué tan fácil es entender un texto: valores altos indican lenguaje claro y accesible, mientras que valores bajos señalan mayor complejidad. Por su parte, el score de factuality mide la fidelidad del contenido generado respecto al texto original: valores cercanos a 1 indican alta coherencia factual y valores bajos posibles desviaciones o errores en la información.

Los resultados muestran un comportamiento coherente entre los modelos pequeños y los modelos de gran escala.

Entre los modelos ajustados localmente, *Llama-3.2-1B* y *Qwen-3-1.7B* alcanzaron las métricas F1 más altas (~0.82) y los mejores niveles de legibilidad relativa, evidenciando una adecuada adaptación al lenguaje sencillo. No obstante, el *AlignScore* promedio (0.28 – 0.34) refleja que los modelos pequeños aún presentan limitaciones en la factualidad de los contenidos generados.

Por su parte, los modelos grandes (Claude Sonnet 4.5 y Grok 4) superaron ampliamente a los modelos locales en factualidad (0.52 – 0.54) y mantuvieron una legibilidad similar o mejor, lo que sugiere una mayor capacidad de razonamiento y preservación semántica sin sacrificar claridad.

En conjunto, los resultados confirman que el fine-tuning local en GPUs de gama media permite obtener modelos competitivos en legibilidad y coherencia, mientras que la factualidad y consistencia semántica se benefician significativamente del uso de modelos de gran escala y mayor capacidad de cómputo.

5. Análisis de los resultados obtenidos. Reflexión sobre los retos encontrados y cómo proseguir

5.1. Clasificador

Si bien las diferencias de rendimiento son mínimas, el modelo contextual requirió mucho mayor tiempo de entrenamiento y una capacidad de cómputo significativamente superior, además de tener un proceso de inferencia más lento. Esto se debe al procesamiento secuencial de tokens y al mayor costo de las capas atencionales en comparación con la vectorización TF-IDF.

El modelo disperso representa la mejor relación entre precisión, interpretabilidad y costo computacional, siendo ideal para la clasificación de grandes volúmenes de texto científico y PLS. El modelo contextual, aunque competitivo, se recomienda principalmente para aplicaciones más complejas donde la semántica contextual aporte un valor diferencial.

5.2. Fine tuning

La primera iteración demuestra que, con fine-tuning local, los modelos pequeños igualan la relevancia/coherencia, pero la factualidad aún requiere técnicas adicionales (RAG, verificación, decodificación guiada). Migrar el entrenamiento a A100 permitirá mejor exploración de hiperparámetros y datasets, integrando controles de longitud y verificadores para cerrar la brecha de factualidad sin perder legibilidad. Con ello, el prototipo quedará en posición de ofrecer PLS confiables, concisos y verificables para el dominio de salud.

Cabe resaltar que, **hasta el momento, no se ha aplicado un análisis con técnicas de Chain-of-Thought (CoT)** para los resultados actuales. Este paso podría cambiar los resultados en la siguiente iteración, ya que permitirá examinar el razonamiento interno de los modelos, identificar patrones de error y ajustar los prompts o estrategias de entrenamiento para mejorar la factualidad y la consistencia narrativa.

De cara a la siguiente fase, se planea **migrar el entrenamiento a entornos con GPU A100** (vía Colab) para aprovechar una mayor capacidad de cómputo, incorporar el análisis CoT y evaluar nuevas configuraciones de fine-tuning y métricas complementarias que permitan consolidar la calidad y confiabilidad de los resúmenes generados.

Referencias

Chall, J. S., & Dale, E. (1995). Readability revisited: The new Dale–Chall readability formula. Brookline Books.

Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). ELECTRA: Pre-training text encoders as discriminators rather than generators. In Proceedings of the International Conference on Learning Representations (ICLR 2020). Retrieved from <https://openreview.net/forum?id=r1xMH1BtvB>

Coleman, M., & Liao, T. L. (1975). A computer readability formula designed for machine scoring. Journal of Applied Psychology, 60(2), 283–284. <https://doi.org/10.1037/h0076540>

Flesch, R. (1948). A new readability yardstick. Journal of Applied Psychology, 32(3), 221–233. <https://doi.org/10.1037/h0057532>

Gunning, R. (1952). The technique of clear writing. McGraw–Hill.

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2019). GLUE: A multi-task benchmark and analysis platform for natural language understanding. In Proceedings of the International Conference on Learning Representations (ICLR 2019). Retrieved from <https://openreview.net/forum?id=rJ4km2R5t7>

Zha, Y., Yang, Y., Li, R., & Hu, Z. (2023). AlignScore: Evaluating factual consistency with a unified alignment function. arXiv preprint. <https://doi.org/10.48550/arXiv.2305.16739>

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating text generation with BERT. In International Conference on Learning Representations (ICLR 2020). <https://openreview.net/forum?id=SkeHuCVFDr>